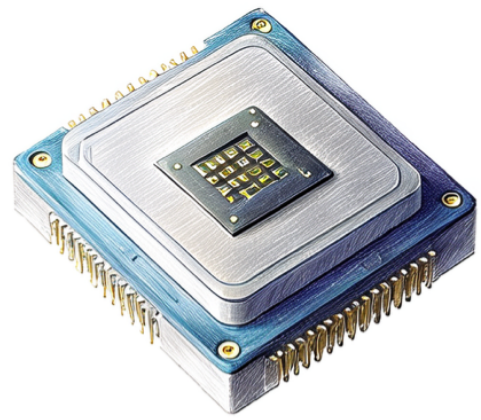


PROCESSOR ARCHITECTURE

Modern microprocessors are amongst the most complex systems ever created.



The instructions supported by a particular processor and their byte-level encodings are known as its ISA.

The ISA provides a conceptual layer of abstraction between compiler writers and chip designers.

NOTE - The way a processor operates can be different from the model of computation implied by the ISA. For example, the ISA can imply sequential execution of all operations in a program. However, a modern processor can execute multiple instructions parallelly to improve performance. Special mechanisms are used to ensure consistency in end results.

→ This chapter uses a freshly constructed Y86-64 ISA

* Defining an ISA -

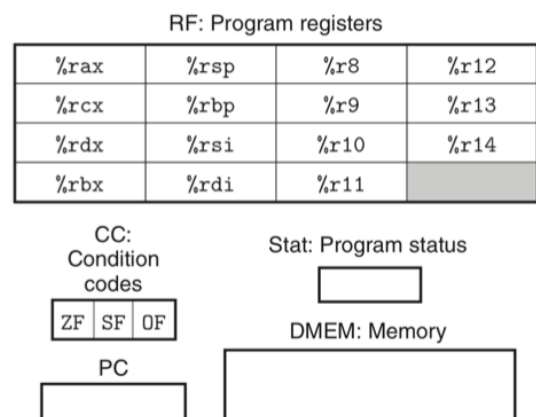
→ This involves defining the various components of its state, the set of instructions and their encodings, a set of programming conventions and the handling of exceptional events.

Each instruction in a Y86-64 program can read and modify some part of the processor state → This is called programmer visible state.

→ assembly code writer or the compiler

The state of Y86-64 is similar to x86-64 -

1. There are 15 program registers.
2. Each register stores a 64 bit word.
3. %rsp is used as the stack pointer.
4. There are three single bit condition codes.
5. The PC holds the currently executed instruction.
6. Memory is a large array of bytes holding program & data
→ Y86-64 programs use virtual addresses.
7. The status code STAT indicates the overall status of execution.



The Y86-64 instruction set -

1. The move instructions - `sd movq`
 - s → represents the source. (can be immediate (i), register (r) or memory (m))
 - d → represents the destination. (can be register (r) or memory (m))
 - memory to memory transfers are disallowed like x86.
2. Integer Operations - `addq, subq, andq, xorq`
 - Only operate on register data
 - x86 allows integer operations directly on memory.

